

Blueprint Técnico y Análisis Funcional del Dashboard Kamaleon/Meleon

1. Visión General del Blueprint Arquitectónico

Basado en la evidencia forense de los dashboards administrativos, el sistema **Kamaleon (Meleon)** opera como un **Orquestador de Movilidad Server-Driven** (Impulsado por el Servidor). A diferencia de las aplicaciones tradicionales donde la lógica reside en el código compilado (APK/IPA), Meleon utiliza una arquitectura donde el comportamiento, la interfaz y los datos se definen en el servidor y se "inyectan" en los dispositivos móviles.

Diagrama de Flujo de Datos (Inferido del Dashboard)

- Capa de Diseño (Input):**
 - **Figma:** Los diseños de UI se importan directamente para generar la estructura visual.
 - **SQL Server (Backend):** Almacena las definiciones JSON de los módulos, templates de tickets y queries de reportes.
- Capa de Orquestación (Meleon Dashboard):**
 - **Feature Builder:** Convierte diseños en "Manifiestos JSON" y los liga a queries SQL.
 - **Guardian IA:** Valida la seguridad del código SQL antes del despliegue.
 - **Middleware Gateway:** Configura disparadores (Triggers) que conectan eventos móviles con APIs externas (SAP, RENIEC).
- Capa de Distribución (Sincronización):**
 - **Sync Engine:** Distribuye paquetes diferenciales (JSON + SQL Scripts) a la flota móvil.
 - **Velocidad:** 45 MB/s promedio mediante protocolos comprimidos.¹
- Capa de Ejecución (Dispositivo Móvil - Offline First):**
 - **Motor de Renderizado:** Interpreta el JSON para dibujar la UI nativa (No Webview).
 - **SQLite Local:** Ejecuta la lógica de negocio y persiste datos sin conexión.

2. Análisis Detallado de Funcionalidades del Dashboard

A continuación se desglosan los módulos funcionales identificados en las capturas, detallando su capacidad técnica y rol en el sistema.

2.1 Feature Builder: El Constructor de Experiencias

Este módulo actúa como un IDE visual para crear aplicaciones móviles sin tocar código nativo (Android/iOS).

- **Integración Figma-to-Code:** La funcionalidad "Importar Figma & Ligar Datos" permite traducir componentes visuales de diseño directamente a controles nativos.¹
- **Binding de Datos SQL:** Los controles de UI (como un Dropdown) no tienen datos estáticos. Se "hidratan" mediante queries SQL ejecutadas localmente en el dispositivo.
 - *Ejemplo:* El dropdown "Forma de Pago" ejecuta SELECT id, descripcion FROM formas_pago directamente sobre la base de datos SQLite del teléfono.¹
- **Inyección de Contexto:** El sistema utiliza variables de entorno como @current_client_type para adaptar la consulta al perfil del usuario logueado, permitiendo que una misma app se comporte distinto para un "Vendedor" que para un "Supervisor".¹

2.2 Middleware Gateway: Orquestación de Eventos

El Gateway es el puente entre el mundo móvil (que a menudo está desconectado) y los sistemas empresariales externos.

- **Triggers de Eventos (Event-Driven):** Permite definir lógica que se dispara ante acciones del usuario.
 - *Caso de Uso:* El trigger ON_EXIT(dni_field) invoca al servicio "RENIEC Lookup" para validar una identidad en tiempo real (145ms de latencia).¹
- **Bloqueo Condicional:** Puede impedir el avance del flujo de trabajo si una validación externa falla (IF response.valid == false THEN Block Submit), asegurando la integridad de los datos en el punto de captura.¹
- **Monitoreo en Vivo:** Un panel de "Live Gateway Log" permite a los ingenieros diagnosticar fallos de integración (ej. Timeouts con SAP) en tiempo real.¹

2.3 Diseñador de Tickets (Motor de Impresión)

Este módulo resuelve la complejidad de la impresión térmica en campo (Bluetooth 80mm).

- **Templating JSON:** Los tickets no son imágenes, son estructuras JSON ("type": "header", "type": "items") que el dispositivo renderiza. Esto ahorra ancho de banda y permite cambios de formato instantáneos.¹
- **Inyección de Datos Dinámicos:** Los campos del ticket como {{tenant_name}} o {{pedido.total}} se rellenan en el momento de la impresión usando los datos de la transacción local.¹
- **Código QR Generativo:** Capacidad de generar códigos QR al vuelo ("type": "qr_code") para facturación electrónica o validación de entregas.¹

2.4 Cerebros AI & Agentes (RAG Engine)

Meleon incorpora una capa de inteligencia artificial para soporte y automatización.

- **RAG (Retrieval-Augmented Generation):** Permite cargar documentos corporativos (PDFs como politica_precios_2025.pdf) que se vectorizan para que los agentes de IA puedan responder preguntas de negocio específicas del cliente ("¿Cuál es el descuento

máximo?").¹

- **Integración n8n:** Los flujos de trabajo de IA se orquestan mediante **n8n**, lo que sugiere una alta capacidad de personalización para conectar la IA con otros sistemas (ej. enviar un email de resumen o alertar a un supervisor).¹

2.5 Guardian IA: Seguridad Preventiva

Dado que el sistema permite desplegar SQL a dispositivos remotos, la seguridad es crítica.

- **Análisis de Intención SQL:** Guardian IA analiza los scripts (Triggers, Queries) antes de que se distribuyan. Detecta y bloquea comandos destructivos como DROP TABLE (identificado como "MOMOP TABLE" en los logs de rechazo).¹
- **Validación de Lógica de Negocio:** Verifica que los Triggers cumplan con las políticas, como la regla de aprobación para pedidos mayores a \$1000.¹

3. Diseño Técnico del Sistema (Arquitectura de Datos)

El análisis del "Modelo de Datos Móvil" revela una arquitectura **Local-First** robusta basada en SQLite.¹

3.1 Esquema de Datos Distribuido

El dispositivo móvil contiene una réplica funcional de la base de datos empresarial, optimizada para el rendimiento local:

Tabla (Entidad)	Rol en el Sistema	Estrategia de Sincronización
DEVICE_ORDER	Transaccional (Escritura)	Alta Frecuencia (5m): Crítico para asegurar que los pedidos lleguen al ERP casi en tiempo real.
DEVICE_PRODUCT	Catálogo (Lectura)	Media (2h / Versión): Catálogos grandes que cambian poco. Se sincronizan por lotes para ahorrar batería.
DEVICE_CUSTOMER	Maestro (Lectura/Escritura)	Diaria: Información base de clientes, rutas y saldos.

DEVICE_ORDERTAXITEM	Lógica Fiscal	Automática: Motores de cálculo de impuestos que residen en el dispositivo para garantizar exactitud offline.
---------------------	---------------	---

3.2 Estructura del Manifiesto de UI

El diseño técnico de la UI se basa en un contrato JSON estricto que desacopla la presentación de la lógica:¹

JSON

```
{
  "component": "dropdown",
  "id": "cbo_pago",
  "data_source": {
    "type": "sqlite",
    "query": "SELECT id, descripcion FROM formas_pago WHERE activo = 1...",
    "params": ["@client_type"]
  }
}
```

Este diseño permite cambiar las opciones del dropdown (ej. quitar "Cheque" como forma de pago) simplemente actualizando la consulta SQL en el servidor, sin actualizar la app en la tienda.

4. Plan de Implementación y Despliegue

Basado en las herramientas de "Release (CI/CD)" y "Gestión Multi-Tenant" del dashboard, se propone el siguiente flujo de implementación para un nuevo cliente:

Fase 1: Configuración del Tenant (Día 1-2)

1. **Alta en Multi-Tenant:** Crear la instancia del cliente en el orquestador ("Retail Premium").¹
2. **Definición de Perfiles:** Configurar roles (Vendedor, Supervisor) y asignar los módulos

base (Pedidos, Cobranza).¹

3. **Carga de Datos Maestros:** Inyectar los datos iniciales (Clientes, Productos) para que el motor de sincronización genere la primera base de datos SQLite (db.sql).

Fase 2: Diseño y Lógica (Día 3-10)

1. **Importación UI:** Utilizar el **Feature Builder** para importar los diseños de Figma de las pantallas de toma de pedido.¹
2. **Binding SQL:** Conectar cada campo de la UI (precios, stocks, descuentos) a las tablas locales DEVICE_PRODUCT y DEVICE_PRODUCT_PRICE.¹
3. **Reglas de Negocio:** Configurar **Triggers** en el Gateway para validaciones críticas (ej. Consulta de Crédito en SAP).¹
4. **Validación Guardian:** Ejecutar **Guardian IA** para asegurar que todos los scripts SQL y Triggers son seguros y performantes.¹

Fase 3: Despliegue y Estabilización (Día 11-15)

1. **Release CI/CD (Beta):** Desplegar la versión "Pedidos V3.0" a un grupo piloto de dispositivos usando la funcionalidad de Release.¹
2. **Monitoreo Telemetría:** Vigilar el dashboard de Telemetría para asegurar que la "Tasa de Errores" se mantenga por debajo del 0.05% y la velocidad de sync sea óptima (45MB/s).¹
3. **Ajuste de RAG:** Cargar manuales de venta y políticas actualizadas en el "Cerebro AI" para que el asistente de soporte responda dudas de los vendedores en campo.¹

Fase 4: Operación Continua

- **Actualizaciones Zero-Touch:** Para cambios menores (ej. cambiar el formato del Ticket de Venta), editar el JSON en el "Diseñador de Tickets" y sincronizar. Los dispositivos recibirán el cambio en menos de 5 minutos sin intervención del usuario.¹

5. Conclusión del Análisis

El dashboard de **Meleon** revela una plataforma madura que resuelve el problema fundamental de la movilidad empresarial: **la rigidez**. Al mover la definición de la UI y la lógica de datos al servidor (y sincronizarlas como metadatos), Mobility Apps Inc. ha creado un sistema que permite iterar productos digitales a la velocidad de la web, pero con la robustez y capacidad offline de una app nativa. La inclusión de validación por IA (Guardian) y motores de orquestación (Gateway/n8n) sugiere una arquitectura preparada para escalar a operaciones de misión crítica de gran volumen.

Fuentes citadas

1. [screenshot-mobilityappsinc-mx-kamaleon-orquestador-demo-html-2026-01](#)

-04-12_26_12.pdf